

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №8 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Создание и запуск задач»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 7

Вопросы для защиты работы **9**

Выводы **10**

Постановка задачи

Цель данной лабораторной работы: изучить механизм работы с классами задач.

Задачи лабораторной работы:

- Изучить возможности класса Task
- Научиться создавать и запускать задачи
- Научиться работать с массивами потоков

Индивидуальное задание

Написание программного кода

Создадим шаблон консольного проекта при помощи утилиты `dotnet`:

```
dotnet new console -n tasks_101
```

В корне проекта создастся файл Project.cs, в котором и будет написан код лабораторной работы.

Создаем класс Program с методом Main:

```
class Program
{
        static void Main(string[] args)
        {

        }
}
```

Для того, чтобы передать `Task` некоторую задачу, определим статический метод `CalcMatrix`, который будет содержать логику, выполняемую асинхронно:

```
static void CalcMatrix(object o)
{
        List<List<int> > matrix = (List<List<int>>)o;
        for (int i = 0; i < MATRIX_SIZE; i++)
        {
                for(int j = 0; j < MATRIX_SIZE; j++)
                {
                        if (matrix[i][j] % 2 == 0) matrix[i][j] = (int)Math.Tan((double
                            )matrix[i][j]);
                }
        }
        Thread.Sleep(1000);
        Console.WriteLine("\Задачап выполнена. Вывод:");
        foreach (var row in matrix)
        {
                foreach (var item in row)
                {
                        Console.Write("{0}, ", item);
                }
                Console.WriteLine();
        }
}
```

Сигнатура метода: принимает параметр типа `object` и возвращает void (в соответствии с сигнатурой делегата). Чтобы передать в функцию матрицу, необходимо сделать приведение типов, в данном случае к `List<List<int>>`.

В методе осуществляется проход по элементам матрицы, и если элемент четный, то он заменяется на его тангенс. В конце проходом циклов `foreach` делается

вывод в консоль, чтобы удостовериться в правильности работы алгоритма.

Опишем функцию `Main`. Она должна содержать:

- Логику генерации матрицы по двум константам: `MATRIX_SIZE` и `MAX_VALUE`
- Вывод в консоль результата работы
- Создание и вызов объекта `Task`.

```
private const int MATRIX_SIZE = 10;
private const int MAX_VALUE = 1000;
static void Main(string[] args)
{
        Random random = new Random();
        Action<object> action = CalcMatrix;
        List<List<int> > matrix = new List<List<int>>();
        for (int i = 0; i < MATRIX_SIZE; i++)
        {
                matrix.Add(new List<int>());
                for (int j = 0; j < MATRIX_SIZE; j++)
                {
                        matrix[i].Add(random.Next(MAX_VALUE));
                }
        }

        Console.WriteLine("Матрица сгенерирована. Вывод:");
        foreach (var row in matrix)
        {
                foreach (var item in row)
                {
                        Console.Write("{0}, ", item);
                }
                Console.WriteLine();
        }

        Console.WriteLine("\Задачап отправлена на выполнение...");
        Task task = new Task(action, matrix);
        task.Start();
        Console.ReadKey();
}
```

Дополнительно сделан вывод по результатам ключевых действий программы.

Итоговый код в листинге ниже:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
```

```
using System.Threading.Tasks;

namespace Tasks_101
{
        internal class Program
        {
                private const int MATRIX_SIZE = 10;
                private const int MAX_VALUE = 1000;
                static void Main(string[] args)
                {
                        Random random = new Random();
                        Action<object> action = CalcMatrix;
                        List<List<int> > matrix = new List<List<int>>();
                        for (int i = 0; i < MATRIX_SIZE; i++)
                        {
                                matrix.Add(new List<int>());
                                for (int j = 0; j < MATRIX_SIZE; j++)
                                {
                                        matrix[i].Add(random.Next(MAX_VALUE));
                                }
                        }

                        Console.WriteLine("Матрица сгенерирована. Вывод:");
                        foreach (var row in matrix)
                        {
                                foreach (var item in row)
                                {
                                        Console.Write("{0}, ", item);
                                }
                                Console.WriteLine();
                        }

                        Console.WriteLine("\Задачап отправлена на выполнение...");
                        Task task = new Task(action, matrix);
                        task.Start();
                        Console.ReadKey();
                }

                static void CalcMatrix(object o)
                {
                        List<List<int> > matrix = (List<List<int>>)o;
                        for (int i = 0; i < MATRIX_SIZE; i++)
                        {
                                for(int j = 0; j < MATRIX_SIZE; j++)
                                {
                                        if (matrix[i][j] % 2 == 0) matrix[i][j] = (int)
                                            Math.Tan((double)matrix[i][j]);
                                }
                        }
                        Thread.Sleep(1000);
                        Console.WriteLine("\Задачап выполнена. Вывод:");
                        foreach (var row in matrix)
```

```
                        {
                                foreach (var item in row)
                                {
                                        Console.Write("{0}, ", item);
                                }
                                Console.WriteLine();
                        }
                }
        }
}
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
dotnet build; dotnet run
```

Вывод программы:

```
Матрица сгенерирована. Вывод:
960, 660, 908, 975, 347, 422, 112, 767, 435, 669,
505, 253, 490, 461, 4, 337, 72, 764, 831, 779,
717, 176, 308, 540, 960, 777, 657, 53, 82, 437,
183, 928, 77, 609, 286, 179, 89, 382, 71, 771,
957, 22, 152, 740, 164, 466, 363, 442, 123, 62,
604, 446, 412, 663, 932, 784, 352, 367, 14, 570,
764, 59, 684, 997, 485, 576, 175, 881, 175, 825,
248, 689, 692, 59, 821, 695, 24, 568, 376, 494,
414, 215, 929, 19, 833, 736, 516, 414, 676, 502,
827, 721, 368, 501, 7, 898, 337, 154, 254, 587,

Задача отправлена на выполнение...

Задача выполнена. Вывод:
-4, 0, 0, 975, 347, 1, -1, 767, 435, 669,
505, 253, 0, 461, 1, 337, 0, 0, 831, 779,
717, 0, 0, 0, -4, 777, 657, 53, 0, 437,
183, 2, 77, 609, 0, 179, 89, -3, 71, 771,
957, 0, 2, -6, 0, 1, 363, -1, 123, -1,
1, 0, 0, 663, -1, -5, 0, 367, 7, 4,
0, 59, -1, 997, 485, 1, 175, 881, 175, 825,
0, 689, 1, 59, 821, 695, -2, 0, -1, 0,
0, 215, 929, 19, 833, 1, 0, 0, 0, 0,
827, 721, 0, 501, 7, 0, 337, 0, 0, 587,
```

Программа выполняется корректно. Можно удостовериться, что все четные элементы матрицы были изменены.

Вопросы для защиты работы

1. **Что такое класс Task? Чем класс Task отличается от класса Thread?**

   Класс `Task` - абстракция над потоками, которая берет на себя всё, что связано с созданием единиц исполнения - `Thread`. Это удобно, потому что программист не заботится о регулировании количества потоков и ресурсах, которые с ними связаны.

2. **Как получить идентификатор текущей задачи? Как получить идентификатор текущего потока?**

   Идентификатор задачи - с помощью статического свойства `Task.CurrentId`. Идентификатор потока - с помощью `Thread.CurrentThread.ManagedThreadId`.

3. **Опишите возможные методы создания и запуска задач.**

   - С помощью класса `Task` и вызовом метода `Start`
   - С помощью фабрики - класса `ThreadFactory`

4. **Какие параметры задачи может установить программист при реализации задачи? Опишите возможные варианты применения параметра типа `TaskCreationOptions`.**

   - Запуск синхронной задачи через `RunSynchronously`
   - `TaskCreationOptions` для указания ”важности”задачи. `LongRunning` - под задачу по возможности выделяется поток. `PreferFairness` - задача низкоприоритетная. `AttachToParent`, если задача должна быть присоединена к родительской

Выводы

В данной лабораторной работе изучен способ работы с потоками, абстрагирующий программиста от их внутренней реализации, при помощи класса `Task`. Были показаны способы работы с ним, в том числе через конструктор класса.